

NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

College of Computing and Data Science

SC2006 – Software Engineering

Lab 2 Deliverables – AI Tech Stack Recommendation

SCSJ Group 3

Name	Matriculation No.
Xu Hanwen	U2522613H
Teo Kok Wei, Oscar	U2410894D
K Eswaran	U2521677A
Mao Yikai	U2520028F
Tan Leong Jun Joseph	U2321339H

(1) Description of uploaded material

Uploaded Crowdy's Lab 1 system description (a Singapore-only real-time crowd-checking app aggregating data from LTA DataMall and Google Popular Times, with GPS-based map discovery, user accounts, and favourites) and the Lab 2 class diagram of entity classes (Venue, Station, Account, Favourite, MapRegion, with their attributes and relationships).

(2) Prompt and AI response

Prompt:

Based on the uploaded design materials, recommend a technology stack appropriate for an entry-level software engineering team.

Response:

Based on your requirements — a cross-platform mobile app (iOS/Android/Web from one codebase), a backend aggregating multiple external data sources, MySQL for accounts/favourites, and password security via one-way hashing — here's a stack suited to an entry-level team:

Frontend: React Native (with Expo) for cross-platform delivery from a single JS/TypeScript codebase, satisfying your portability requirement directly. Flutter is a reasonable alternative but introduces Dart as a new language; React Native has a lower learning curve if your team already knows JavaScript/React.

Backend: Node.js with Express. Keeping the whole stack in JavaScript/TypeScript minimizes context-switching for students learning full-stack development, and Express's middleware model fits your requirement for authentication checks and a swappable data-source layer.

Database: MySQL, as specified, via an ORM such as Prisma or Sequelize to reduce hand-written SQL boilerplate for a small team.

Caching layer: Redis, to hold cached results from your external data sources (station crowd data, mall crowd estimates) with automatic expiry, decoupling your cache-freshness logic from application code and letting multiple backend instances share cache state.

Authentication: JWT-based sessions with bcrypt password hashing, matching your stated security requirement.

External data integration: A dedicated adapter/service per data source (e.g. a scraping service using a headless browser for sources without an official API), so sources can be added or swapped without touching the rest of the system.

Hosting: A managed platform (e.g. Render or Railway) for the backend, and a managed MySQL host (e.g. PlanetScale or Aiven) for the database, avoiding infrastructure management for a small team.

Testing/CI: Jest for backend unit tests, React Native Testing Library for frontend, and GitHub Actions for CI to run linting/type-checking/tests on every push.

(3) Written opinion

We chose not to use Redis as our caching needs (5-min/daily windows, single backend instance) are met by a simple in-memory cache at no extra operational cost. Redis would make sense at production scale with multiple instances, not for a course project.